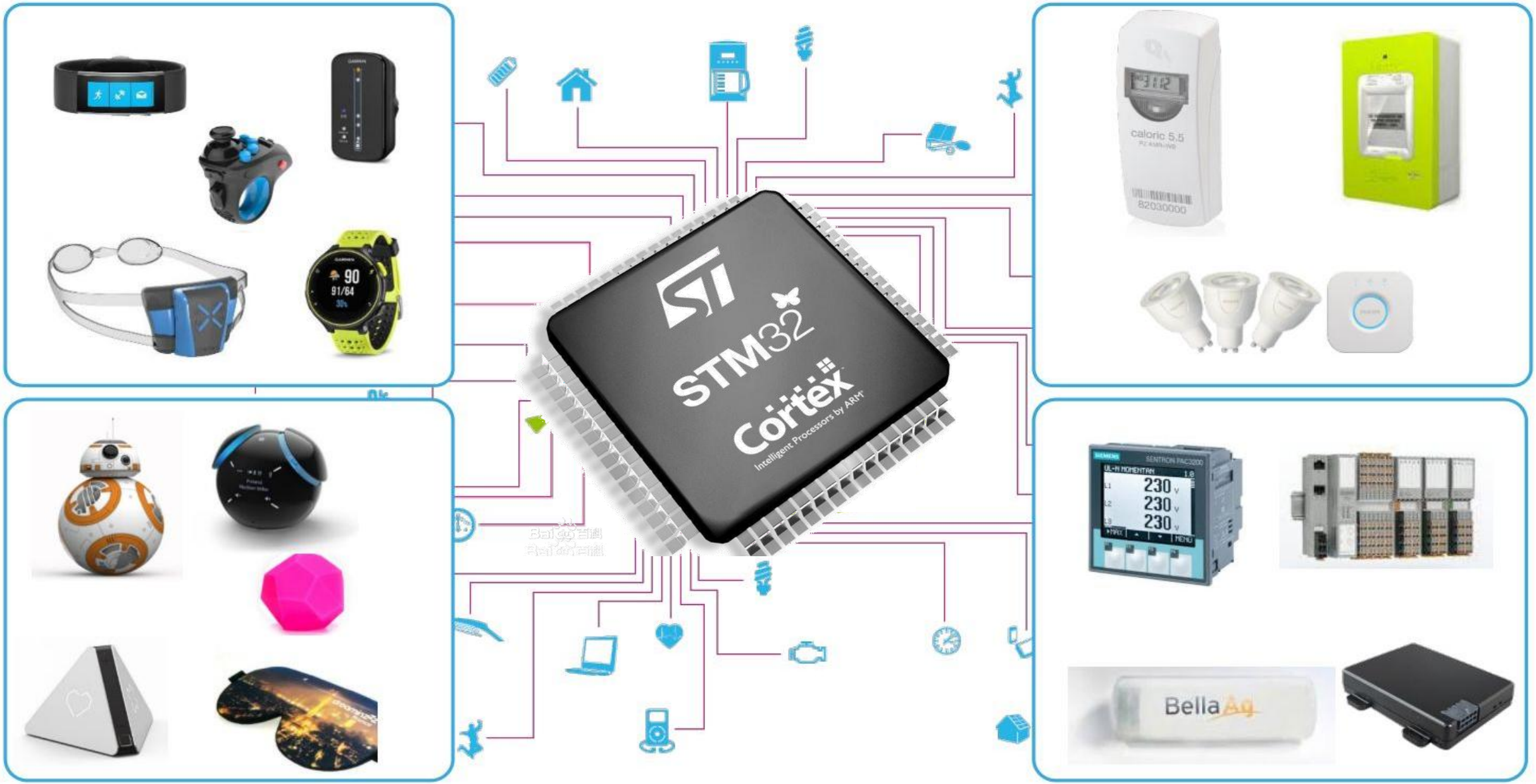
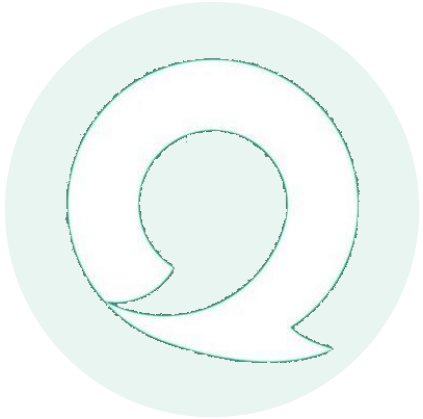


Instructor：WangXiaojing Email：wangxj@sustech.edu.cn



Lecture5

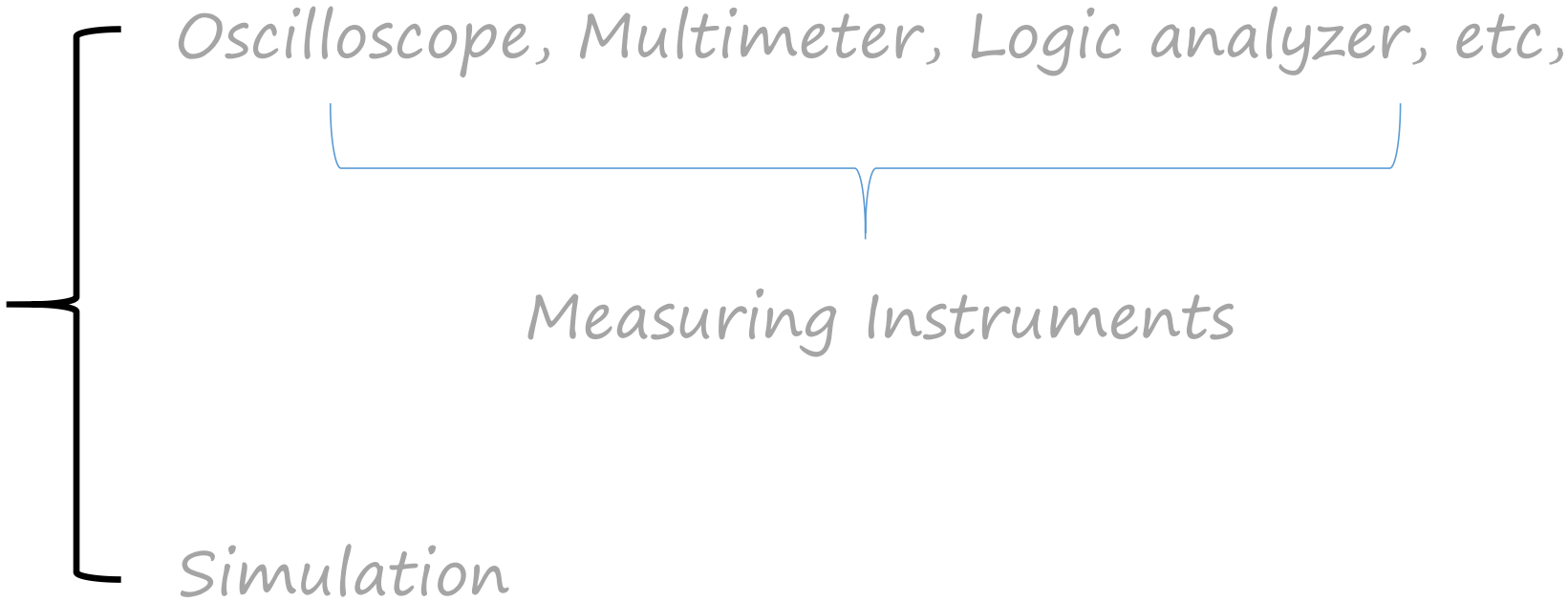
Debug & Clocks & NVIC

Topics

- Debug
- Clocks
- NVIC--Interrupt Control

Part I : Debug

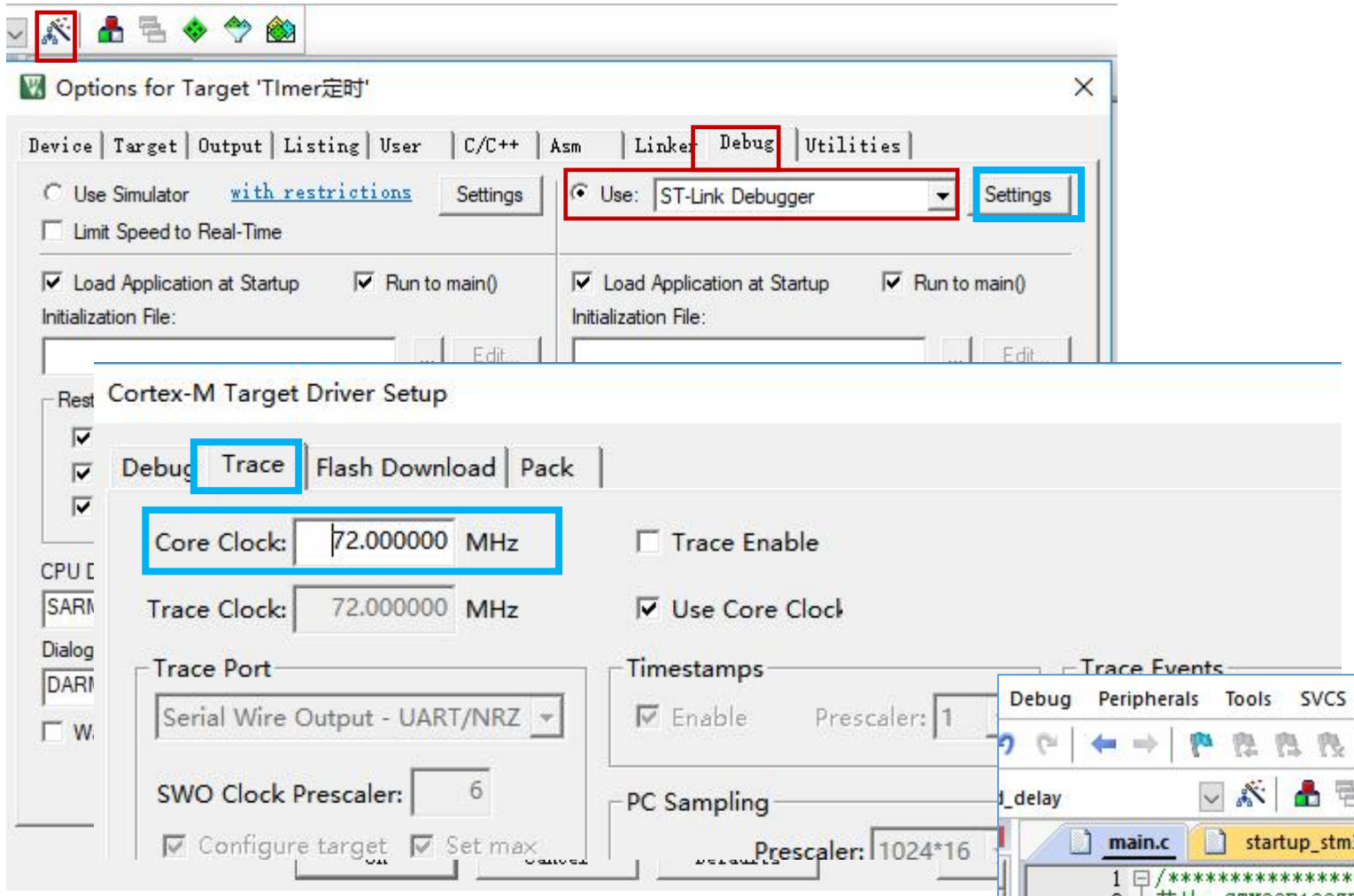
Debug

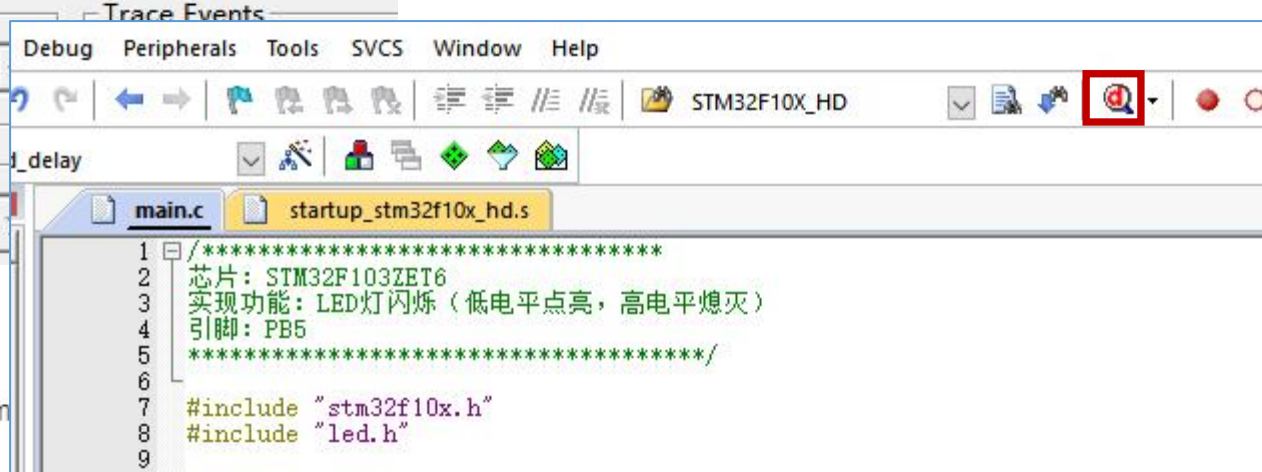

Oscilloscope, Multimeter, Logic analyzer, etc,
Measuring Instruments
Simulation

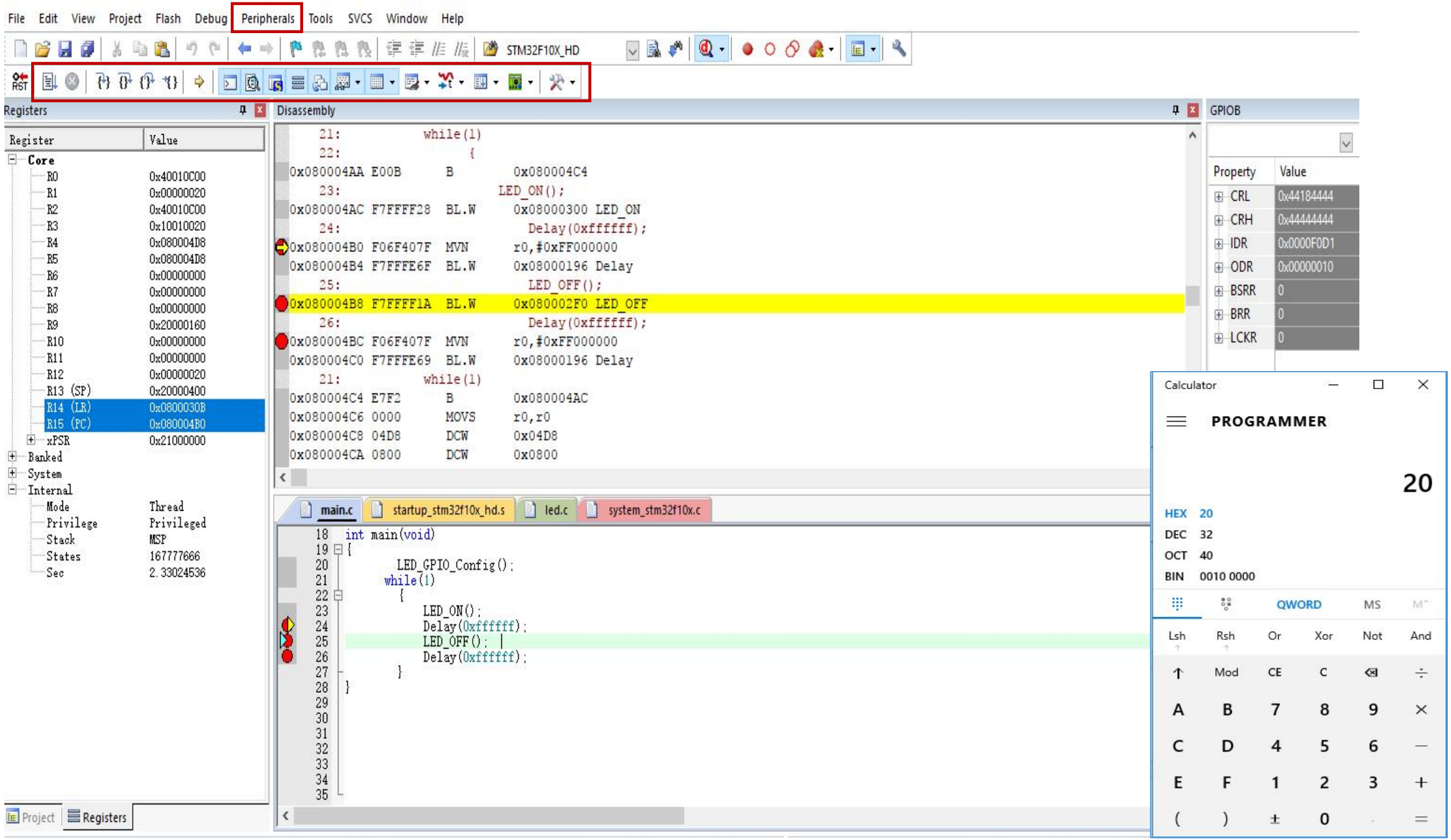
Simulation


Hardware simulation
with physical objects
Software simulation
without physical objects

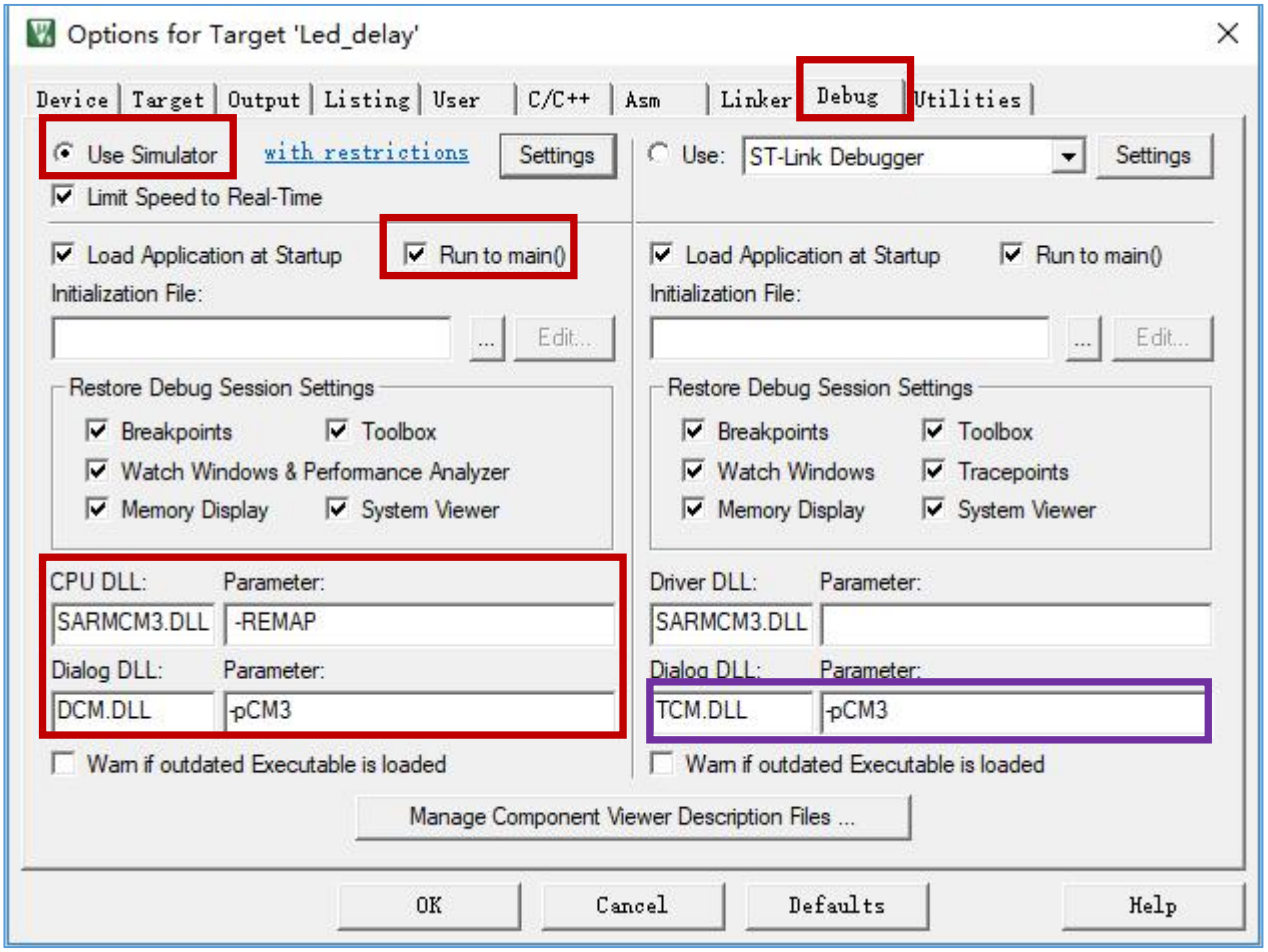
Hardware simulation

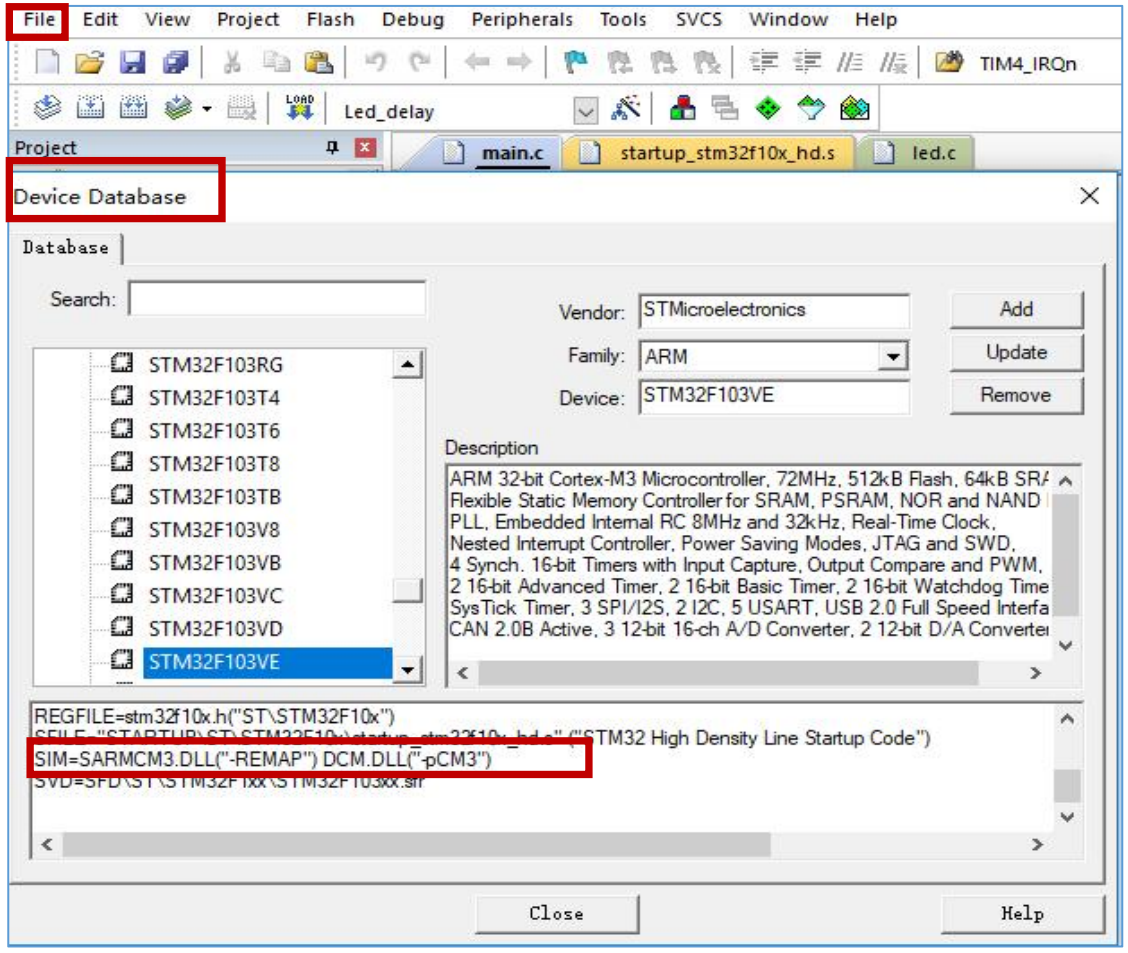





File Edit View Project Flash Debug Peripherals Tools SVCS Window Help
STM32F10X_HD
Registers
Register Value
Core
R0 0x40010C00
R1 0x00000020
R2 0x40010C00
R3 0x10010020
R4 0x080004D8
R5 0x080004D8
R6 0x00000000
R7 0x00000000
R8 0x00000000
R9 0x20000160
R10 0x00000000
R11 0x00000000
R12 0x00000020
R13 (SP) 0x20000400
R14 (LR) 0x0800030B
R15 (PC) 0x080004B0
xPSR 0x21000000
Banked
System
Internal
Mode Thread
Privilege Privileged
Stack MSP
States 167777666
Sec 2.33024536
Project
Registers
Disassembly
21: while(1)
22: {
0x080004AA E00B B 0x080004C4
23: LED_ON();
0x080004AC F7FFFF28 BL.W 0x08000300 LED_ON
24: Delay(0xffffff);
0x080004B0 F06F407F MVN r0,#0xFF000000
0x080004B4 F7FFFE6F BL.W 0x08000196 Delay
25: LED_OFF();
0x080004B8 F7FFFF1A BL.W 0x080002F0 LED_OFF
26: Delay(0xffffff);
0x080004BC F06F407F MVN r0,#0xFF000000
0x080004C0 F7FFFE69 BL.W 0x08000196 Delay
21: while(1)
0x080004C4 E7F2 B 0x080004AC
0x080004C6 0000 MOVS r0,r0
0x080004C8 04D8 DCW 0x04D8
0x080004CA 0800 DCW 0x0800
main.c
startup_stm32f10x_hd.s
led.c
system_stm32f10x.c
18 int main(void)
19 {
20 LED_GPIO_Config();
21 while(1)
22 {
23 LED_ON();
24 Delay(0xffffff);
25 LED_OFF();
26 Delay(0xffffff);
27 }
28 }
GPIOB
Property Value
CRL 0x44184444
CRH 0x44444444
IDR 0x0000F0D1
ODR 0x00000010
BSRR 0
BRR 0
LCKR 0
Calculator
PROGRAMMER
20
HEX 20
DEC 32
OCT 40
BIN 0010 0000
QWORD
MS
Lsh Rsh Or Xor Not And
Mod CE C
A B 7 8 9
C D 4 5 6
E F 1 2 3
( ) ± 0

Software simulation

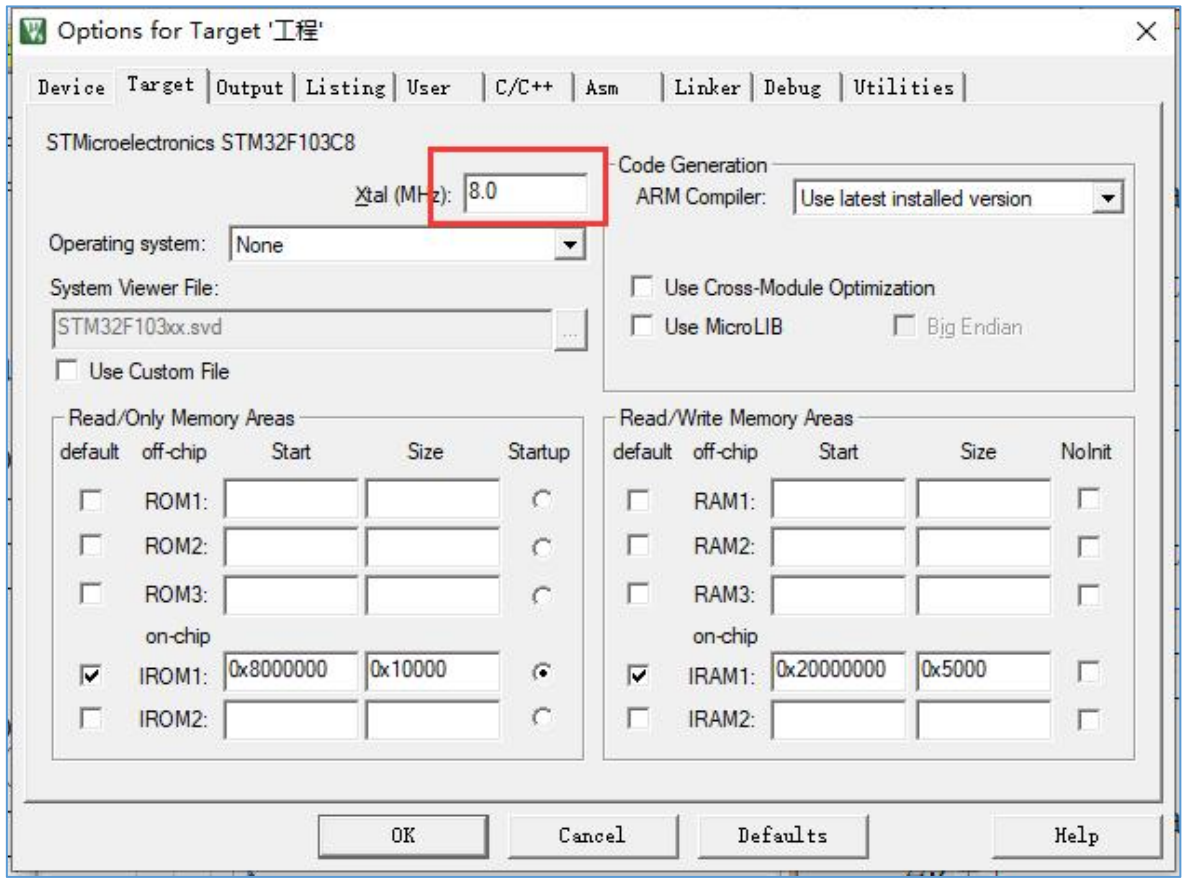


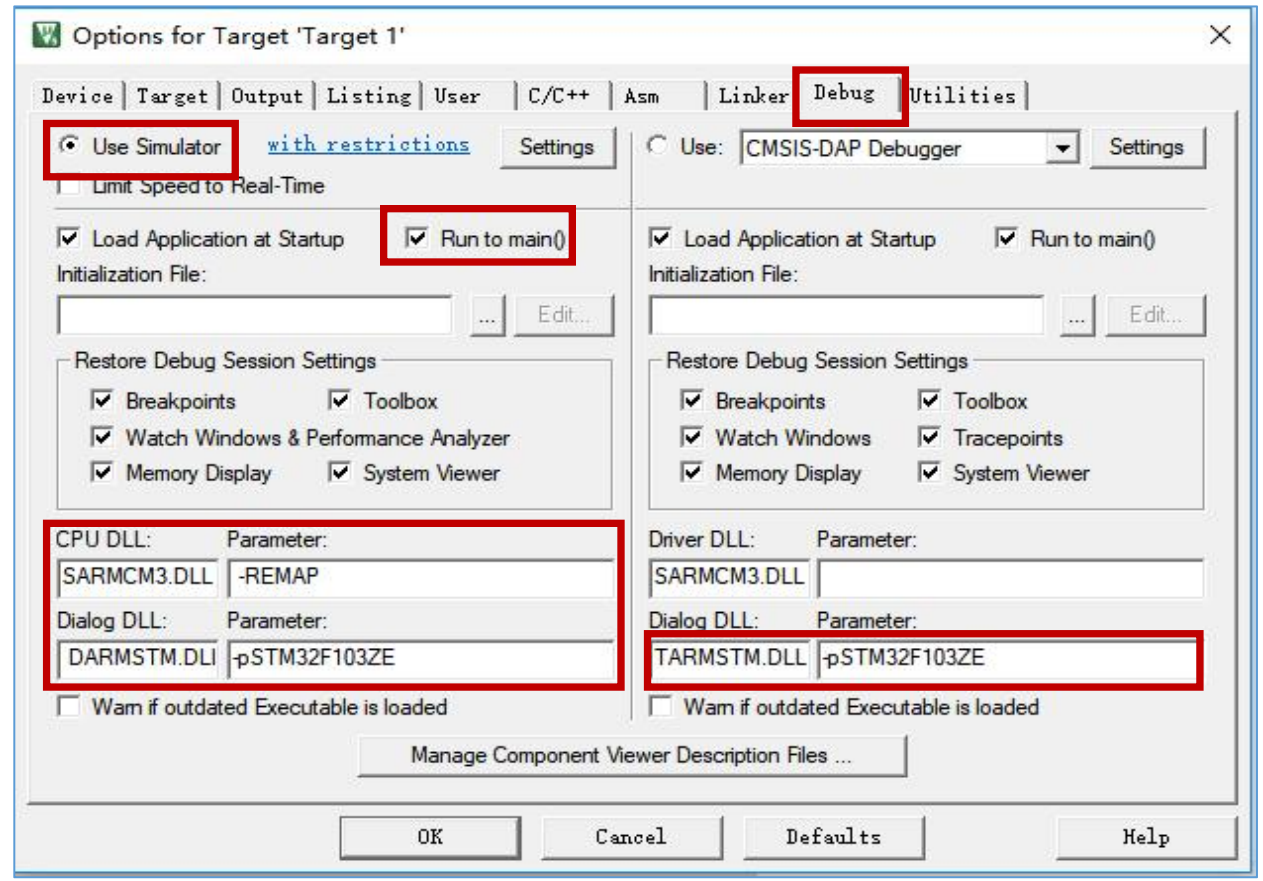


https://www.keil.com/support/man/docs/uv4/uv4_dg_debug.htm

CPU/Driver DLL - Parameter

Preferably, do not modify these settings. Device Database Parameters describe the parameters.





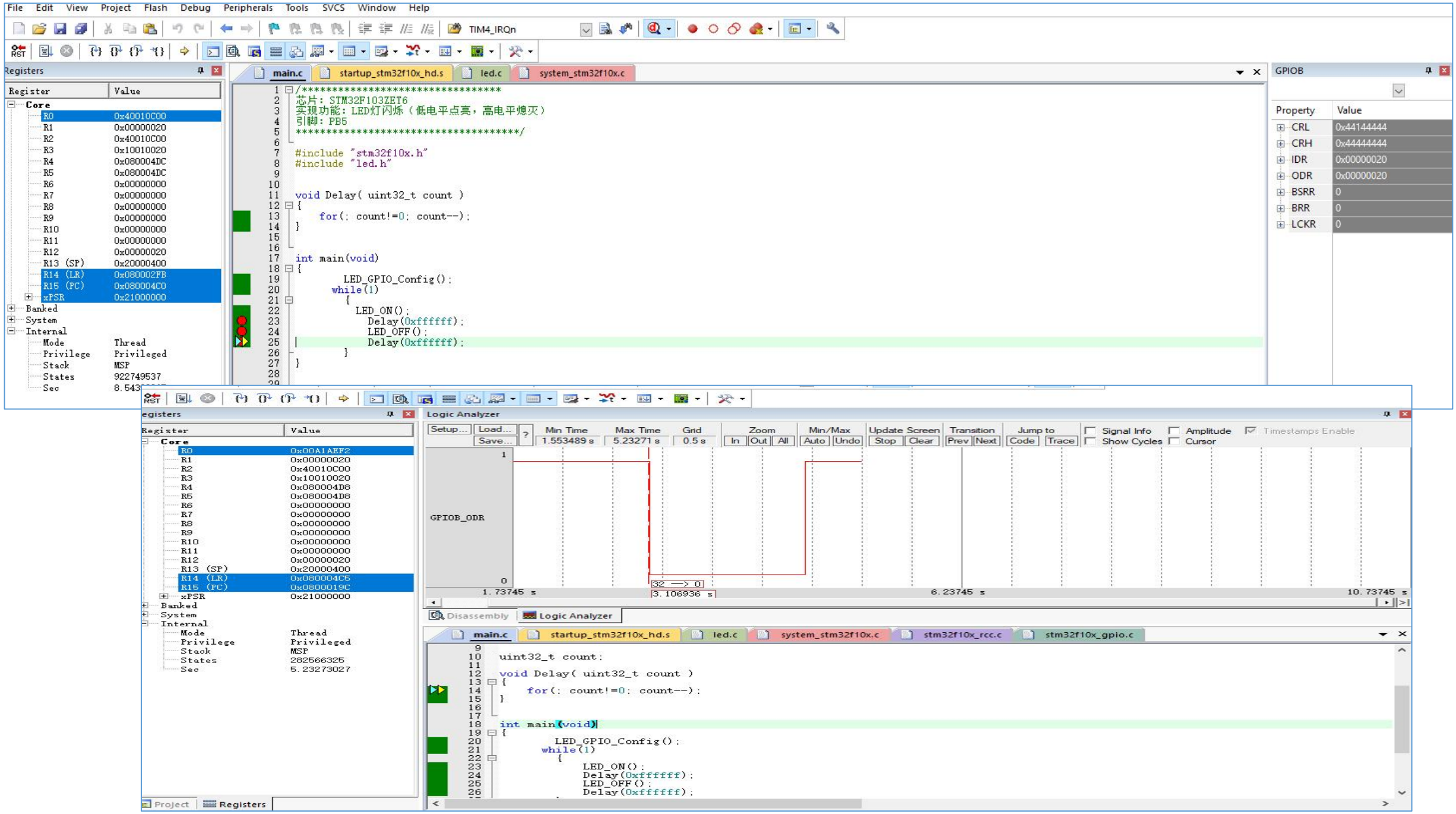
https://developer.arm.com/documentation/ka002225/latest

Peripheral Simulation for STMicroelectronics STM32F1 Series

Dialog DLL	Parameter	Microcontroller Device
DARMSTM.DLL	-pSTM32F101C6	STM32F101C6
DARMSTM.DLL	-pSTM32F101C8	STM32F101C8
DARMSTM.DLL	-pSTM32F101CBT6	STM32F101CBT6
DARMSTM.DLL	-pSTM32F101R6	STM32F101R6
DARMSTM.DLL	-pSTM32F101R8	STM32F101R8
DARMSTM.DLL	-pSTM32F101RB	STM32F101RB
DARMSTM.DLL	-pSTM32F101T6	STM32F101T6
DARMSTM.DLL	-pSTM32F101T8	STM32F101T8
DARMSTM.DLL	-pSTM32F101V8	STM32F101V8
DARMSTM.DLL	-pSTM32F101VB	STM32F101VB
DARMSTM.DLL	-pSTM32F103C6	STM32F103C6
DARMSTM.DLL	-pSTM32F103C8	STM32F103C8
DARMSTM.DLL	-pSTM32F103CB	STM32F103CB
DARMSTM.DLL	-pSTM32F103R6	STM32F103R6
DARMSTM.DLL	-pSTM32F103R8	STM32F103R8
DARMSTM.DLL	-pSTM32F103RB	STM32F103RB
DARMSTM.DLL	-pSTM32F103T6	STM32F103T6
DARMSTM.DLL	-pSTM32F103T8	STM32F103T8
DARMSTM.DLL	-pSTM32F103V8	STM32F103V8
DARMSTM.DLL	-pSTM32F103VB	STM32F103VB
DARMSTM.DLL	-pSTM32F103ZE	STM32F103ZE

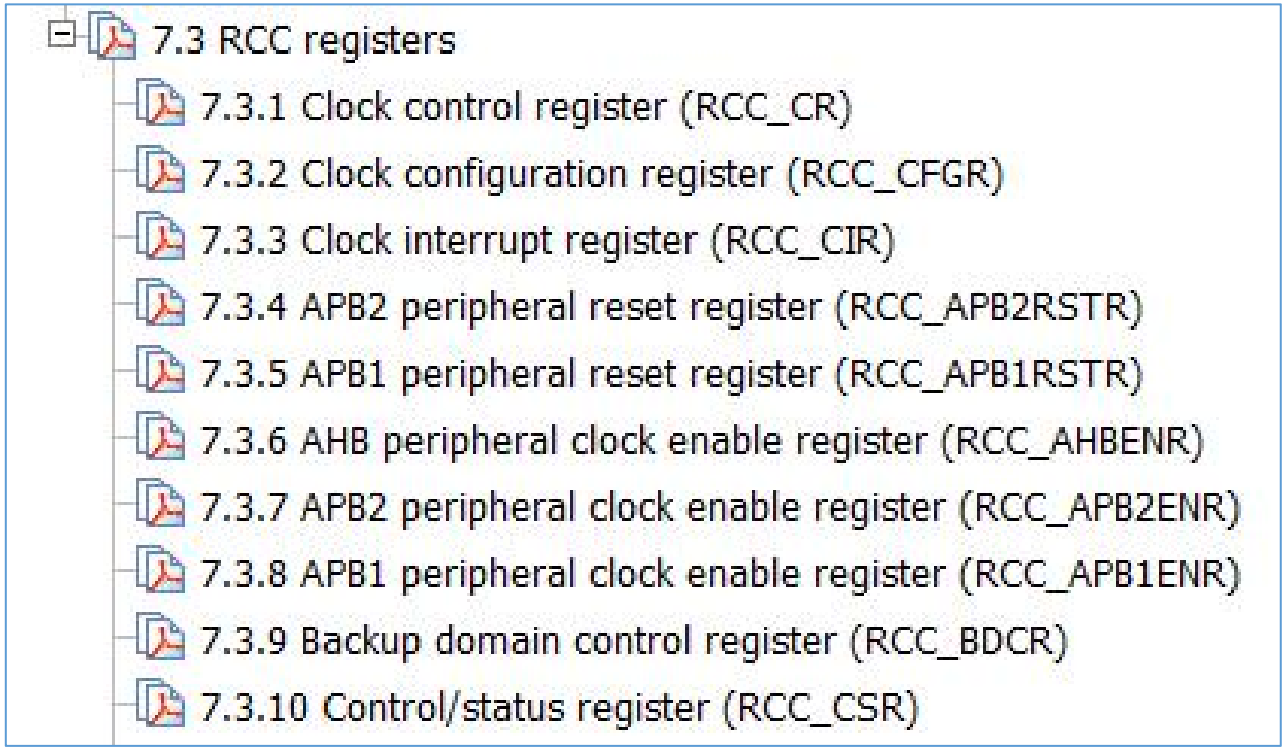
Peripheral Simulation for NXP LPC1100 Series

Dialog DLL	Parameter	Microcontroller Device


File Edit View Project Flash Debug Peripherals Tools SVCS Window Help
TIM4_IRQn
Registers
Register Value
Core
R0 0x40010C00
R1 0x00000020
R2 0x40010C00
R3 0x10010020
R4 0x080004DC
R5 0x080004DC
R6 0x00000000
R7 0x00000000
R8 0x00000000
R9 0x00000000
R10 0x00000000
R11 0x00000000
R12 0x00000020
R13 (SP) 0x20000400
R14 (LR) 0x080002FB
R15 (PC) 0x080004C0
xPSR 0x21000000
Banked
System
Internal
Mode Thread
Privilege Privileged
Stack MSP
States 922749537
Sec 8.543
main.c
startup_stm32f10x_hd.s
led.c
system_stm32f10x.c
/**********
芯片：STM32F103ZET6
实现功能：LED灯闪烁（低电平点亮，高电平熄灭）
引脚：PB5
**********/
#include "stm32f10x.h"
#include "led.h"
void Delay(uint32_t count)
{
for(; count!=0; count--);
}
int main(void)
{
LED_GPIO_Config();
while(1)
{
LED_ON();
Delay(0xffffff);
LED_OFF();
Delay(0xffffff);
}
}
GPIOB
Property Value
CRL 0x44144444
CRH 0x44444444
IDR 0x00000020
ODR 0x00000020
BSRR 0
BRR 0
LCKR 0
Logic Analyzer
Setup... Load... Save...
Min Time 1.553489 s
Max Time 5.23271 s
Grid 0.5 s
Zoom In Out All
Min/Max Auto Undo
Update Screen Stop Clear
Transition Prev Next
Jump to Code Trace
Signal Info
Show Cycles
Amplitude
Cursor
Timestamps Enable
GPIOB_ODR
1
0
32 —> 0
1.73745 s
3.106936 s
6.23745 s
10.73745 s
Disassembly
Logic Analyzer
stm32f10x_rcc.c
stm32f10x_gpio.c
uint32_t count;
Project
Registers

We can clearly know the running process of the program by debug.

Part II : Clocks



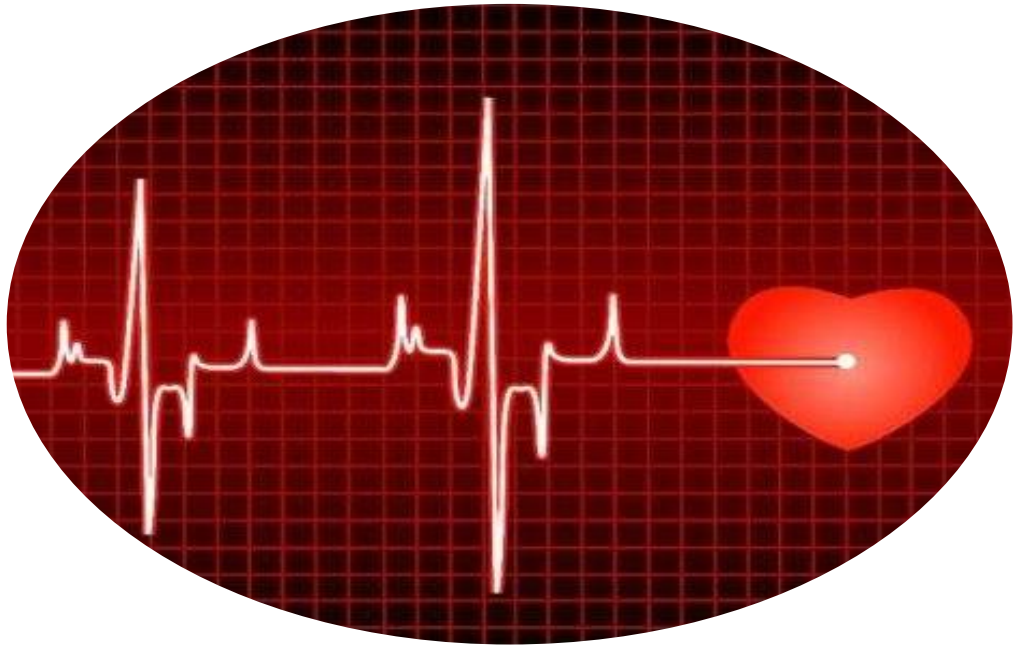
Source

Frequency

```
// 打开 GPIOB 端口的时钟
        *( unsigned int * )0x40021018 |=  ( (1) << 3 );

// 打开 GPIOB 端口的时钟
        RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB,ENABLE);
```

The importance of Clocks



STM32

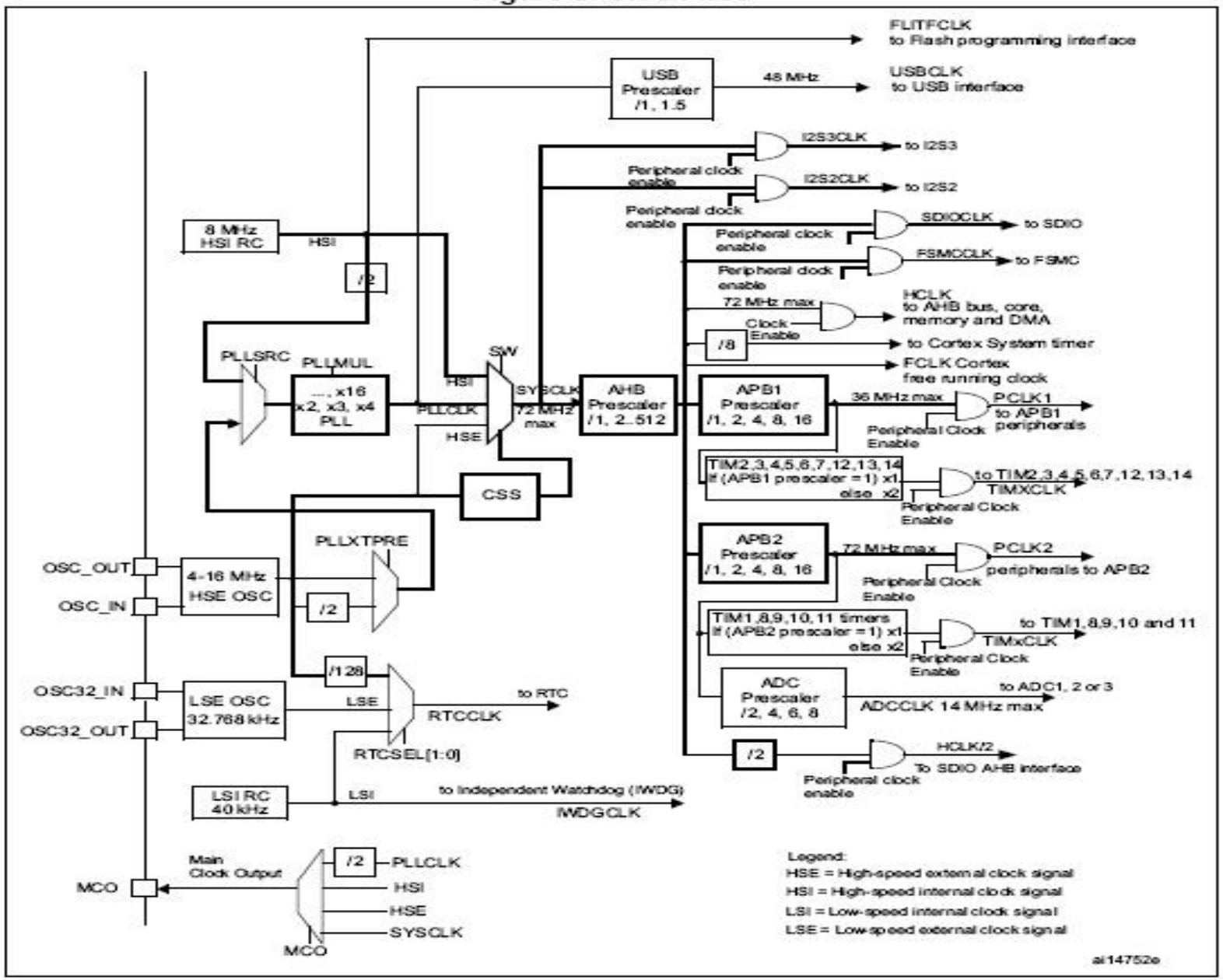
- Low power consumption
- Only enable the clock of the peripheral that we need
- And also, we're able to configure the peripheral clock frequency

peripheral A	peripheral B
72M	8M

startup.s --- void SystemInit(void)

Clocks Tree

Figure 8. Clock tree



1. When the HSI is used as a PLL clock input, the maximum system clock frequency that can be achieved is 64 MHz.
2. For full details about the internal and external clock source characteristics refer to the "Electrical characteristics" section in your device datasheet.

Three different clock sources can be used to drive the system clock (SYSCLK):

- **HSI** oscillator clock
- **HSE** oscillator clock
- **PLL** clock

The devices have the following two secondary clock sources:

- 40 kHz low speed internal RC (**LSI** RC) which drives the independent watchdog and optionally the RTC used for Auto-wakeup from Stop/Standby mode.
- 32.768 kHz low speed external crystal (**LSE** crystal) which optionally drives the real-time clock (RTCCLK)

Each clock source can be switched on or off independently when it is not used, to optimize power consumption.

HSE	H:High	S:Speed	E:External
HSI	H:High	S:Speed	I:Inernal
LSI	L:Low	S:Speed	I:Inernal
LSE	L:Low	S:Speed	E:External
PLL	PLL frequency doubling output		

在STM32F1中，有五个时钟源，为HSE、 HSI、 LSI、 LSE、 PLL。

①HSE: High Speed External 是高速外部时钟，可接石英/陶瓷谐振器，或者接外部时钟源，频率范围为4MHz~16MHz。（我们一般选择8MHz）

②HSI: High Speed Internal 是高速内部时钟，RC振荡器，频率为8MHz。

③LSI: Low Speed Internal 是低速内部时钟，RC振荡器，频率为40kHz。

④LSE: Low Speed External是低速外部时钟，接频率为32.768kHz的石英晶体。

⑤PLL: Phase Lock Loop 为锁相环倍频输出，其时钟输入源可选择为HSI/2、HSE或者HSE/2。倍频可选择为2~16倍，但是其输出频率最大不得超过72MHz。

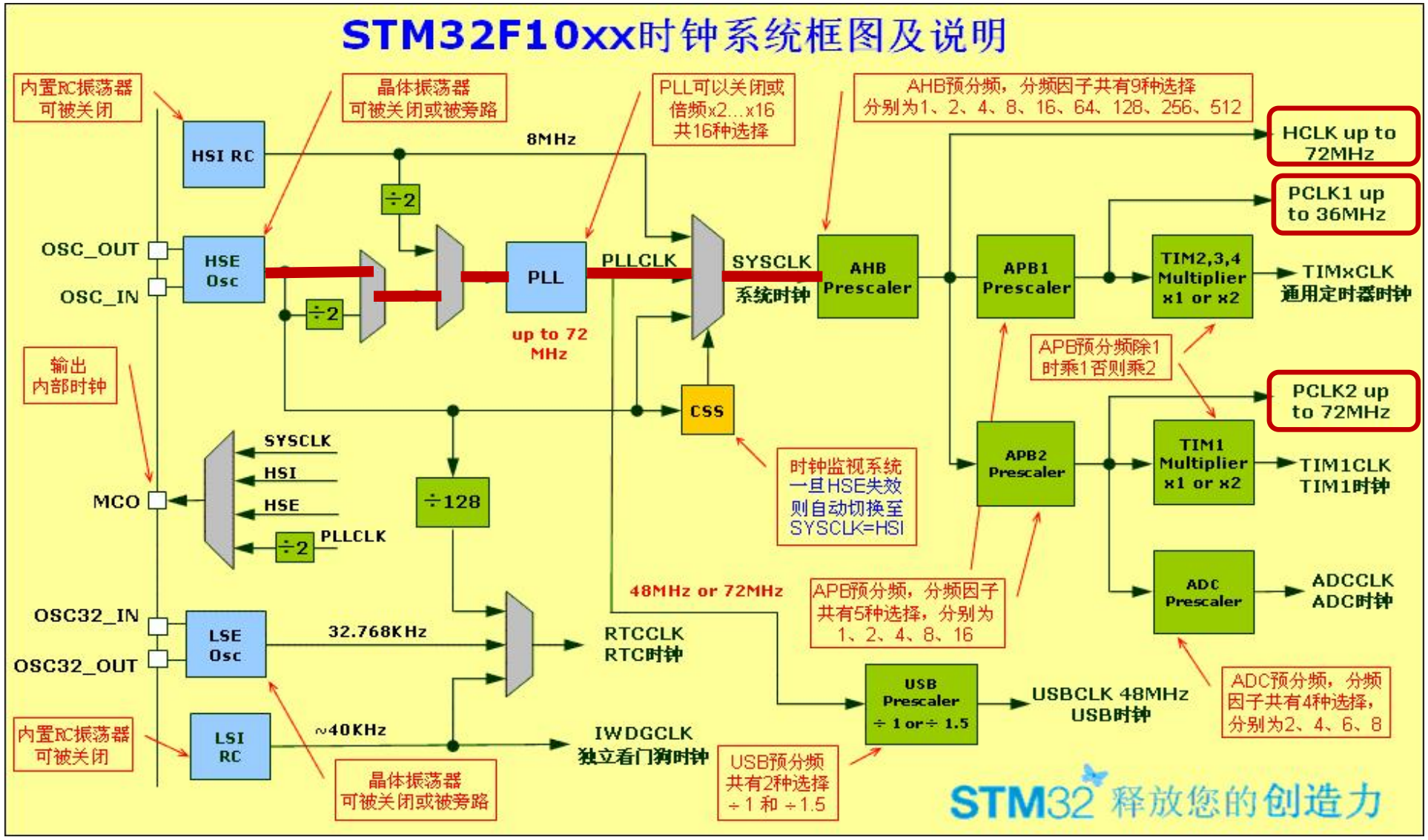
HSE

HSI

LSI

LSE

PLL



蓝色框：时钟源　　灰色框：选择器　　绿色框：倍频或者分频

Summary

1：SYSCLK：系统时钟，最大频率为72MHz，它是供STM32中绝大部分部件工作的时钟源。系统时钟可由PLL、HSI或者HSE提供；

2：HCLK ：AHB总线时钟，由系统时钟SYSCLK 分频得到，一般不分频，等于系统时钟。HCLK是高速外设时钟，提供给存储器、DMA及Cortex内核，是Cortex内核运行的时钟，CPU主频就是这个信号，它的大小与STM32运算速度、数据存取速度密切相关；

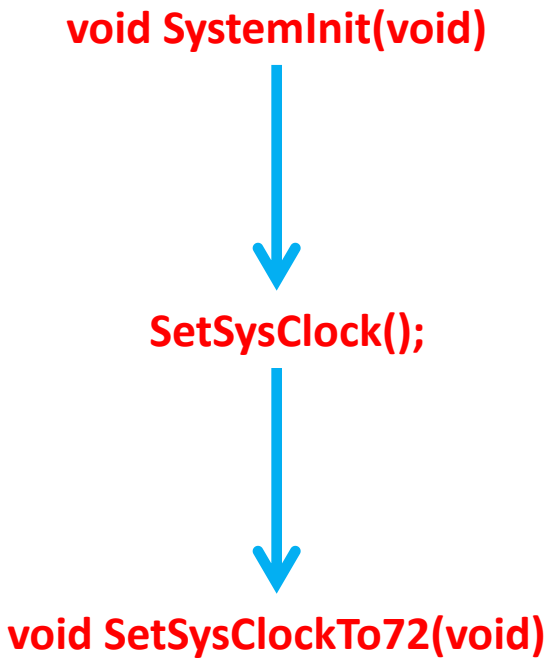
3：FCLK：Cortex®-M3 的”自由运行时钟“。”自由“表现在它不来自于HCLK，因此HCLK时钟停止时FCLK也会继续运行；

4：PCIK1：外设时钟，由APB1预分频器输出得到，最大频率为36MHz，提供给挂载在APB1总线上的外设（低速外设）；

5：PCLK2：外设时钟，由APB2预分频器输出得到，最大频率为72MHz，提供给挂载在APB2总线上的外设（高速外设）；

6：RTC (Real Time Clock)：实时时钟；

7：IWDGCLK(Independent WatchDog Clock)：独立看门狗时钟；



STM32的时钟树是需要配置的，系统时钟即SYSCLK，是供STM32中绝大部分部件工作的时钟源，它可选择为PLL输出、HSI或者HSE（一般程序中采用PLL倍频到72Mhz），系统时钟的最大值Max=72MHz。当系统时钟确定，还需留意：

1）输入时钟源到达外设处的速率没有固定关系，可根据需要进行配置；

2）外设时钟默认不开启，以节省功耗。故使用前必须先要使能其时钟；

Modify the System Clocks to 36M

startup_stm32f10x_hd.s

```
147
148  ; Reset handler
149  Reset_Handler    PROC
150                   EXPORT  Reset_Handler             [WEAK]
151                   IMPORT  __main
152                   IMPORT  SystemInit
153                   LDR     R0, =SystemInit
154                   BLX     R0
155                   LDR     R0, =__main
156                   BX      R0
157                   ENDP
158
```

Download-->Observe the flashing cycle of LED

Hardware simulation & Software Simulation

main.c

```
/**********************************
芯片：STM32F103ZET6
实现功能：LED灯闪烁（低电平点亮，高电平熄灭）
引脚：PB5
**********************************/
#include "stm32f10x.h"
#include "led.h"
void Delay( uint32_t count )
{
        for(; count!=0; count--);
}
int main(void)
{
        LED_GPIO_Config();
        while(1)
                {
                        LED_ON();
                        Delay(0xffffff);

                        LED_OFF();
                        Delay(0xffffff);
                }
}
```

Part III : The NVIC and Interrupt Control

What's Interrupts?

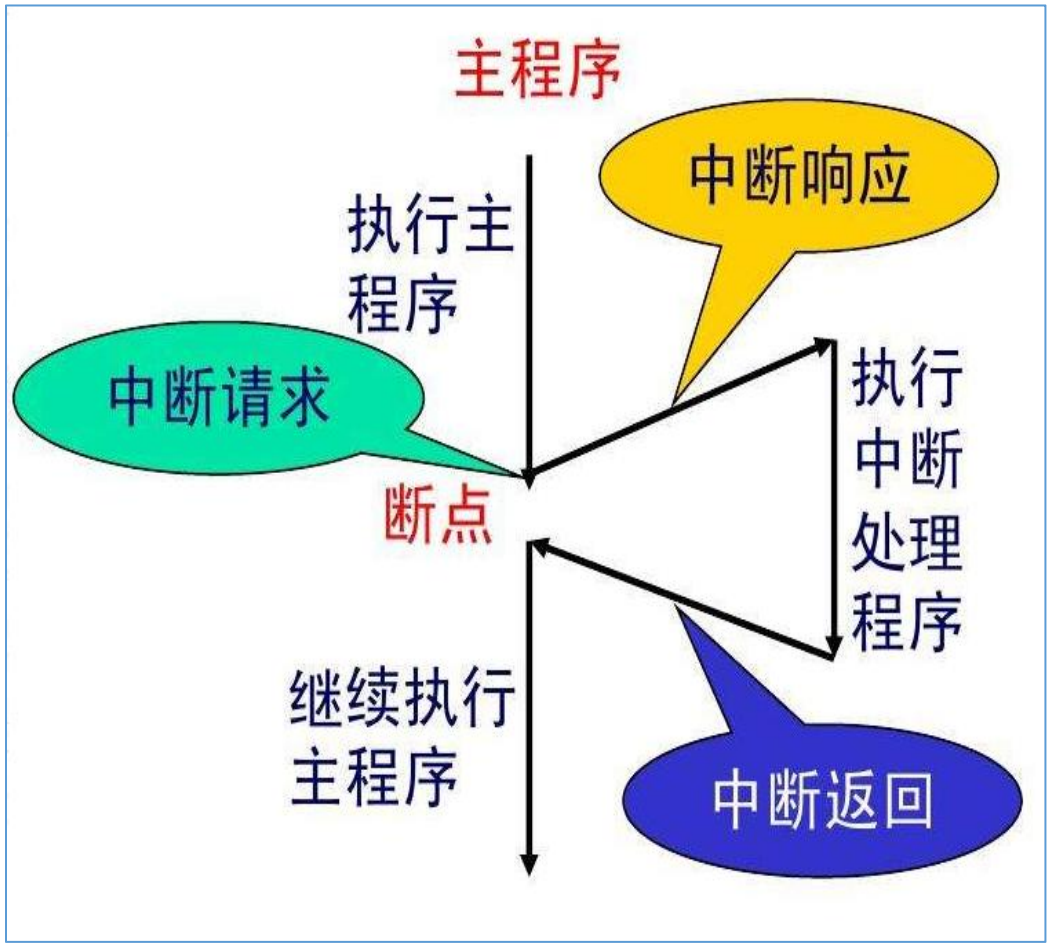


Table 63. Vector table for other STM32F10xxx devices

Position	Priority	Type of priority	Acronym	Description	Address
-	-	-	-	Reserved	0x0000_0000
-	-3	fixed	Reset	Reset	0x0000_0004
-	-2	fixed	NMI	Non maskable interrupt. The RCC Clock Security System (CSS) is linked to the NMI vector.	0x0000_0008
-	-1	fixed	HardFault	All class of fault	0x0000_000C
-	0	settable	MemManage	Memory management	0x0000_0010
-	1	settable	BusFault	Prefetch fault, memory access fault	0x0000_0014
-	2	settable	UsageFault	Undefined instruction or illegal state	0x0000_0018
-	-	-	-	Reserved	0x0000_001C - 0x0000_002B
-	3	settable	SVCall	System service call via SWI instruction	0x0000_002C
-	4	settable	Debug Monitor	Debug Monitor	0x0000_0030
-	-	-	-	Reserved	0x0000_0034
-	5	settable	PendSV	Pendable request for system service	0x0000_0038
-	6	settable	SysTick	System tick timer	0x0000_003C
0	7	settable	WWDG	Window watchdog interrupt	0x0000_0040
1	8	settable	PVD	PVD through EXTI Line detection interrupt	0x0000_0044
2	9	settable	TAMPER	Tamper interrupt	0x0000_0048
3	10	settable	RTC	RTC global interrupt	0x0000_004C
4	11	settable	FLASH	Flash global interrupt	0x0000_0050
5	12	settable	RCC	RCC global interrupt	0x0000_0054
6	13	settable	EXTI0	EXTI Line0 interrupt	0x0000_0058
7	14	settable	EXTI1	EXTI Line1 interrupt	0x0000_005C
8	15	settable	EXTI2	EXTI Line2 interrupt	0x0000_0060
9	16	settable	EXTI3	EXTI Line3 interrupt	0x0000_0064
10	17	settable	EXTI4	EXTI Line4 interrupt	0x0000_0068
11	18	settable	DMA1_Channel1	DMA1 Channel1 global interrupt	0x0000_006C
12	19	settable	DMA1_Channel2	DMA1 Channel2 global interrupt	0x0000_0070
13	20	settable	DMA1_Channel3	DMA1 Channel3 global interrupt	0x0000_0074
14	21	settable	DMA1_Channel4	DMA1 Channel4 global interrupt	0x0000_0078
15	22	settable	DMA1_Channel5	DMA1 Channel5 global interrupt	0x0000_007C
16	23	settable	DMA1_Channel6	DMA1 Channel6 global interrupt	0x0000_0080
17	24	settable	DMA1_Channel7	DMA1 Channel7 global interrupt	0x0000_0084
18	25	settable	ADC1_2	ADC1 and ADC2 global interrupt	0x0000_0088
19	26	settable	USB_HP_CAN_TX	USB High Priority or CAN TX interrupts	0x0000_008C
20	27	settable	USB_LP_CAN_RX0	USB Low Priority or CAN RX0 interrupts	0x0000_0090
21	28	settable	CAN_RX1	CAN RX1 interrupt	0x0000_0094
22	29	settable	CAN_SCE	CAN SCE interrupt	0x0000_0098
23	30	settable	EXTI9_5	EXTI Line[9:5] interrupts	0x0000_009C
24	31	settable	TIM1_BRK	TIM1 Break interrupt	0x0000_00A0
25	32	settable	TIM1_UP	TIM1 Update interrupt	0x0000_00A4
26	33	settable	TIM1_TRG_COM	TIM1 Trigger and Commutation interrupts	0x0000_00A8
27	34	settable	TIM1_CC	TIM1 Capture Compare interrupt	0x0000_00AC
28	35	settable	TIM2	TIM2 global interrupt	0x0000_00B0
29	36	settable	TIM3	TIM3 global interrupt	0x0000_00B4
30	37	settable	TIM4	TIM4 global interrupt	0x0000_00B8
31	38	settable	I2C1_EV	I^2C1 event interrupt	0x0000_00BC
32	39	settable	I2C1_ER	I^2C1 error interrupt	0x0000_00C0
33	40	settable	I2C2_EV	I^2C2 event interrupt	0x0000_00C4
34	41	settable	I2C2_ER	I^2C2 error interrupt	0x0000_00C8
35	42	settable	SPI1	SPI1 global interrupt	0x0000_00CC
36	43	settable	SPI2	SPI2 global interrupt	0x0000_00D0
37	44	settable	USART1	USART1 global interrupt	0x0000_00D4
38	45	settable	USART2	USART2 global interrupt	0x0000_00D8
39	46	settable	USART3	USART3 global interrupt	0x0000_00DC
40	47	settable	EXTI15_10	EXTI Line[15:10] interrupts	0x0000_00E0
41	48	settable	RTCAlarm	RTC alarm through EXTI line interrupt	0x0000_00E4
42	49	settable	USBWakeup	USB wakeup from suspend through EXTI line interrupt	0x0000_00E8
43	50	settable	TIM8_BRK	TIM8 Break interrupt	0x0000_00EC
44	51	settable	TIM8_UP	TIM8 Update interrupt	0x0000_00F0
45	52	settable	TIM8_TRG_COM	TIM8 Trigger and Commutation interrupts	0x0000_00F4
46	53	settable	TIM8_CC	TIM8 Capture Compare interrupt	0x0000_00F8
47	54	settable	ADC3	ADC3 global interrupt	0x0000_00FC
48	55	settable	FSMC	FSMC global interrupt	0x0000_0100
49	56	settable	SDIO	SDIO global interrupt	0x0000_0104
50	57	settable	TIM5	TIM5 global interrupt	0x0000_0108
51	58	settable	SPI3	SPI3 global interrupt	0x0000_010C
52	59	settable	UART4	UART4 global interrupt	0x0000_0110
53	60	settable	UART5	UART5 global interrupt	0x0000_0114
54	61	settable	TIM6	TIM6 global interrupt	0x0000_0118
55	62	settable	TIM7	TIM7 global interrupt	0x0000_011C
56	63	settable	DMA2_Channel1	DMA2 Channel1 global interrupt	0x0000_0120
57	64	settable	DMA2_Channel2	DMA2 Channel2 global interrupt	0x0000_0124
58	65	settable	DMA2_Channel3	DMA2 Channel3 global interrupt	0x0000_0128
59	66	settable	DMA2_Channel4_5	DMA2 Channel4 and DMA2 Channel5 global interrupts	0x0000_012C

System Interrupts

External Interrupts

What's NVIC?

NVIC: the Nested Vectored Interrupt Controller, it is an integrated part of the Cortex-M3 processor.

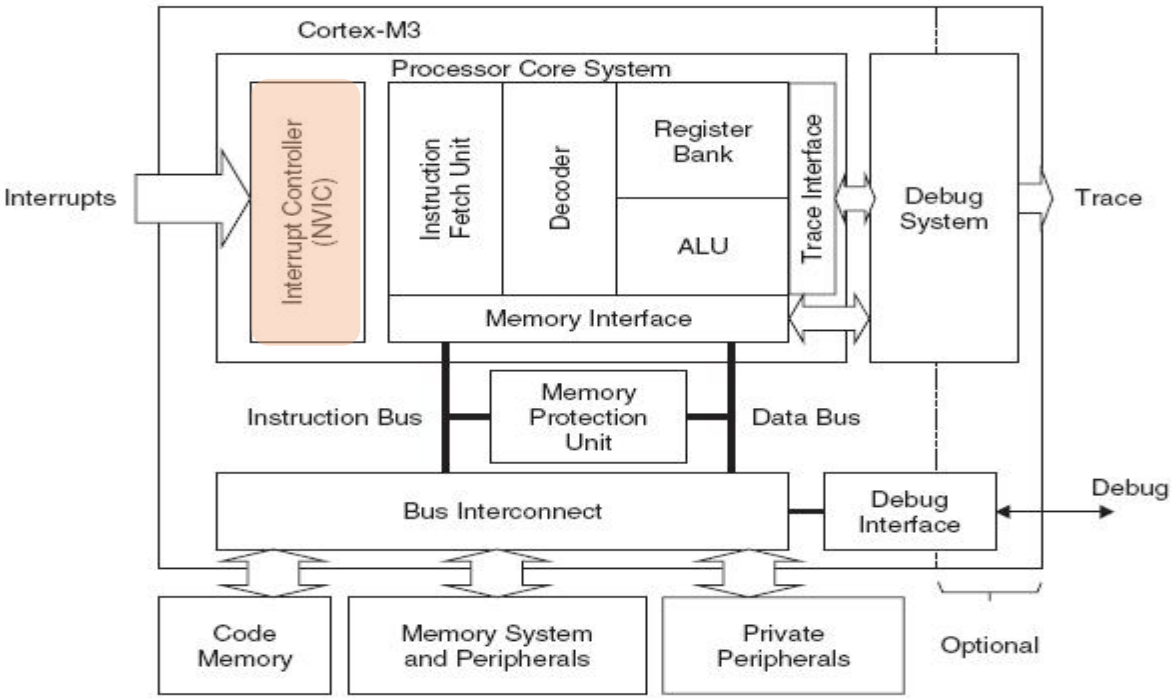


Figure 2.1 A Simplified View of the Cortex-M3

10.1 Nested vectored interrupt controller (NVIC)

Features

- 68 (not including the sixteen Cortex®-M3 interrupt lines)
- 16 programmable priority levels (4 bits of interrupt priority are used)
- Low-latency exception and interrupt handling
- Power management control
- Implementation of System Control registers

The NVIC and the processor core interface are closely coupled, which enables low latency interrupt processing and efficient processing of late arriving interrupts.

All interrupts including the core exceptions are managed by the NVIC. For more information on exceptions and NVIC programming, refer to *STM32F10xxx Cortex®-M3 programming manual* (see *Related documents on page 1*).

4.3.1 The CMSIS mapping of the Cortex®-M3 NVIC registers

To improve software efficiency, the CMSIS simplifies the NVIC register presentation. In the CMSIS:

- The Set-enable, Clear-enable, Set-pending, Clear-pending and Active Bit registers map to arrays of 32-bit integers, so that:
  - The array ISER[0] to ISER[2] corresponds to the registers ISER0-ISER2
  - The array ICER[0] to ICER[2] corresponds to the registers ICER0-ICER2
  - The array ISPR[0] to ISPR[2] corresponds to the registers ISPR0-ISPR2
  - The array ICPR[0] to ICPR[2] corresponds to the registers ICPR0-ICPR2
  - The array IABR[0] to IABR[2] corresponds to the registers IABR0-IABR2.
- The 8-bit fields of the Interrupt Priority Registers map to an array of 8-bit integers, so that the array IP[0] to IP[67] corresponds to the registers IPR0-IPR67, and the array entry IP[n] holds the interrupt priority for interrupt *n*.

PM0056 STM32F10xxx/20xxx/21xxx/L1xxxx Cortex®-M3 programming manual

- 4.3 Nested vectored interrupt controller (NVIC)
  - 4.3.1 The CMSIS mapping of the Cortex®-M3 NVIC registers
  - 4.3.2 Interrupt set-enable registers (NVIC_ISERx)
  - 4.3.3 Interrupt clear-enable registers (NVIC_ICERx)
  - 4.3.4 Interrupt set-pending registers (NVIC_ISPRx)
  - 4.3.5 Interrupt clear-pending registers (NVIC_ICPRx)
  - 4.3.6 Interrupt active bit registers (NVIC_IABRx)
  - 4.3.7 Interrupt priority registers (NVIC_IPRx)
  - 4.3.8 Software trigger interrupt register (NVIC_STIR)

core_cm3.c core_cm3.h
misc.c misc.h

```
127
128 /** @addtogroup CMSIS_CM3_NVIC CMSIS CM3 NVIC
129   memory mapped structure for Nested Vectored Interrupt Controller (NVIC)
130   @{
131  */
132 typedef struct
133 {
134   __IO uint32_t ISER[8];                 /*!< Offset: 0x000  Interrupt Set Enable Register           */
135        uint32_t RESERVED0[24];
136   __IO uint32_t ICER[8];                 /*!< Offset: 0x080  Interrupt Clear Enable Register         */
137        uint32_t RSERVED1[24];
138   __IO uint32_t ISPR[8];                 /*!< Offset: 0x100  Interrupt Set Pending Register          */
139        uint32_t RESERVED2[24];
140   __IO uint32_t ICPR[8];                 /*!< Offset: 0x180  Interrupt Clear Pending Register        */
141        uint32_t RESERVED3[24];
142   __IO uint32_t IABR[8];                 /*!< Offset: 0x200  Interrupt Active bit Register          */
143        uint32_t RESERVED4[56];
144   __IO uint8_t  IP[240];                 /*!< Offset: 0x300  Interrupt Priority Register (8Bit wide) */
145        uint32_t RESERVED5[644];
146   __O  uint32_t STIR;                    /*!< Offset: 0xE00  Software Trigger Interrupt Register     */
147 }  NVIC_Type;
148 /*@}*/ /* end of group CMSIS_CM3_NVIC */
```

core_cm3.h

Interrupt Priority Registers(NVIC_IPRx)

4.3.7 Interrupt priority registers (NVIC_IPRx)

Address offset: 0x00- 0x0B

Reset value: 0x0000 0000

Required privilege: Privileged

The IPR0-IPR16 registers provide a 4-bit priority field for each interrupt. These registers are byte-accessible. Each register holds four priority fields, that map to four elements in the CMSIS interrupt priority array IP[0] to IP[67], as shown in *Figure 19*.

Figure 19. NVIC_IPRx register mapping

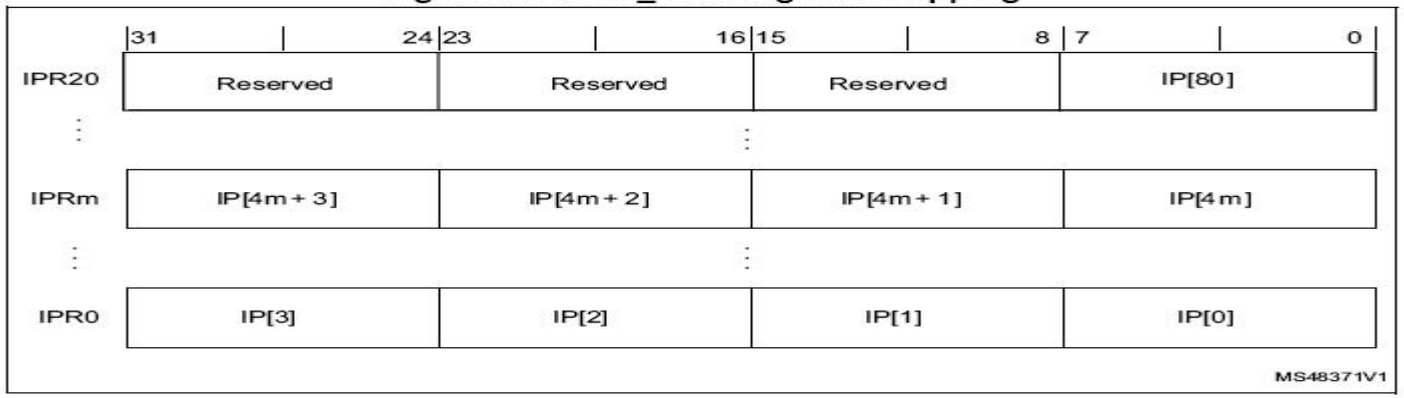


Table 42. IPR bit assignments

Bits	Name	Function
[31:24]	Priority, byte offset 3	Each priority field holds a priority value, 0-255. The lower the value, the greater the priority of the corresponding interrupt. The processor implements only bits[7:4] of each field, bits[3:0] read as zero and ignore writes.
[23:16]	Priority, byte offset 2	
[15:8]	Priority, byte offset 1	
[7:0]	Priority, byte offset 0	

bit7	bit6	bit5	bit4	bit3	bit2	bit1	bit0
用于表达优先级				未使用，读回为 0			

4.4 System control block (SCB)

The *System control block* (SCB) provides system implementation information, and system control. This includes configuration, control, and reporting of the system exceptions. The CMSIS mapping of the Cortex-M3 SCB registers

4.4.5 Application interrupt and reset control register (SCB_AIRCR)

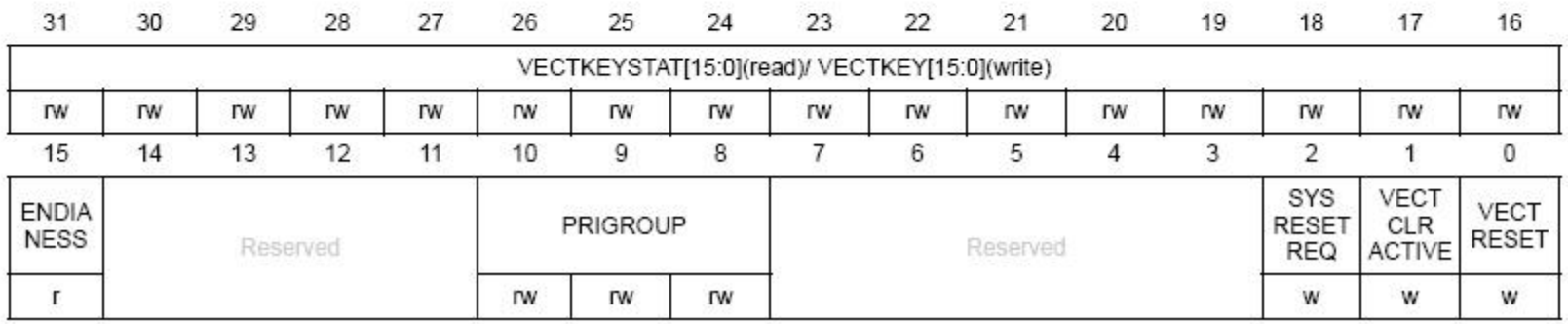
Address offset: 0x0C

Reset value: 0xFA05 0000

Required privilege: Privileged

The AIRCR provides priority grouping control for the exception model, endian status for data accesses, and reset control of the system.

To write to this register, you must write 0x5FA to the VECTKEY field, otherwise the processor ignores the write.



Bits 10:8 **PRIGROUP[2:0]**: Interrupt priority grouping field

This field determines the split of group priority from subpriority, see *Binary point on page 135*.

Table 45. Priority grouping

PRIGROUP [2:0]	Interrupt priority level value, PRI_*N*[7:4]			Number of	
	Binary point[1]	Group priority bits	Subpriority bits	Group priorities	Sub priorities
0b011	0bxxxx	[7:4]	None	16	None
0b100	0bxxx.y	[7:5]	[4]	8	2
0b101	0bxx.yy	[7:6]	[5:4]	4	4
0b110	0bx.yyy	[7]	[6:4]	2	8
0b111	0b.yyyy	None	[7:4]	None	16

1. PRI_*n*[7:4] field showing the binary point. x denotes a group priority field bit, and y denotes a subpriority field bit.

优先级分组

PRIGROUP[2:0]	中断优先级值 PRI_N[7:4]			级数	
	二进制点	主优先级位	子优先级位	主优先级	子优先级
0b 011	0b xxxx	[7:4]	None	16	None
0b 100	0b xxx.y	[7:5]	[4]	8	2
0b 101	0b xx.yy	[7:6]	[5:4]	4	4
0b 110	0b x.yyy	[7]	[6:4]	2	9
0b 111	0b .yyyy	None	[7:4]	None	16

优先级分组	主优先级	子优先级	描述
NVIC_PriorityGroup_0	0	0-15	主-0bit，子-4bit
NVIC_PriorityGroup_1	0-1	0-7	主-1bit，子-3bit
NVIC_PriorityGroup_2	0-3	0-3	主-2bit，子-2bit
NVIC_PriorityGroup_3	0-7	0-1	主-3bit，子-1bit
NVIC_PriorityGroup_4	0-15	0	主-4bit，子-0bit

- 高优先级的主优先级是可以打断正在进行的低主优先级中断的。
- 主优先级相同的中断，高子优先级不可以打断低子优先级的中断。
- 主优先级相同的中断，当两个中断同时发生的情况下，哪个子优先级高，哪个先执行。
- 如果两个中断的主优先级和子优先级都是一样的话，则看哪个中断先发生就先执行。

```c
/**
  * @brief  Configures the priority grouping: pre-emption priority and subpriority.
  * @param  NVIC_PriorityGroup: specifies the priority grouping bits length.
  *   This parameter can be one of the following values:
  *     @arg NVIC_PriorityGroup_0: 0 bits for pre-emption priority
  *                                4 bits for subpriority
  *     @arg NVIC_PriorityGroup_1: 1 bits for pre-emption priority
  *                                3 bits for subpriority
  *     @arg NVIC_PriorityGroup_2: 2 bits for pre-emption priority
  *                                2 bits for subpriority
  *     @arg NVIC_PriorityGroup_3: 3 bits for pre-emption priority
  *                                1 bits for subpriority
  *     @arg NVIC_PriorityGroup_4: 4 bits for pre-emption priority
  *                                0 bits for subpriority
  * @retval None
  */
void NVIC_PriorityGroupConfig(uint32_t NVIC_PriorityGroup)
{
  /* Check the parameters */
  assert_param(IS_NVIC_PRIORITY_GROUP(NVIC_PriorityGroup));

  /* Set the PRIGROUP[10:8] bits according to NVIC_PriorityGroup value */
  SCB->AIRCR = AIRCR_VECTKEY_MASK | NVIC_PriorityGroup;
}
```

misc.c

The table below gives the allowed values of the pre-emption priority and subpriority according to the Priority Grouping configuration performed by NVIC_PriorityGroupConfig function

NVIC_PriorityGroup	NVIC_IRQChannelPreemptionPriority	NVIC_IRQChannelSubPriority	Description
NVIC_PriorityGroup_0	0	0-15	0 bits for pre-emption priority 4 bits for subpriority
NVIC_PriorityGroup_1	0-1	0-7	1 bits for pre-emption priority 3 bits for subpriority
NVIC_PriorityGroup_2	0-3	0-3	2 bits for pre-emption priority 2 bits for subpriority
NVIC_PriorityGroup_3	0-7	0-1	3 bits for pre-emption priority 1 bits for subpriority
NVIC_PriorityGroup_4	0-15	0	4 bits for pre-emption priority 0 bits for subpriority

Summary of Interrupt Control Based on FWLib

- void NVIC_PriorityGroupConfig(uint32_t NVIC_PriorityGroup) ----->Configures the priority grouping,and a project can only be grouped once;

- Configure NVIC related registers--->void NVIC_Init(NVIC_InitTypeDef* NVIC_InitStruct) :(Preemption priority and response priority should be set for each interrupt)

```
50  typedef struct
51  {
52    uint8_t NVIC_IRQChannel;                    /*!< Specifies the IRQ channel to be enabled or disabled.
53                                                     This parameter can be a value of @ref IRQn_Type
54                                                     (For the complete STM32 Devices IRQ Channels list, please
55                                                      refer to stm32f10x.h file) */
56
57    uint8_t NVIC_IRQChannelPreemptionPriority;  /*!< Specifies the pre-emption priority for the IRQ channel
58                                                     specified in NVIC_IRQChannel. This parameter can be a value
59                                                     between 0 and 15 as described in the table @ref NVIC_Priority_Table */
60
61    uint8_t NVIC_IRQChannelSubPriority;         /*!< Specifies the subpriority level for the IRQ channel specified
62                                                     in NVIC_IRQChannel. This parameter can be a value
63                                                     between 0 and 15 as described in the table @ref NVIC_Priority_Table */
64
65    FunctionalState NVIC_IRQChannelCmd;         /*!< Specifies whether the IRQ channel defined in NVIC_IRQChannel
66                                                     will be enabled or disabled.
67                                                     This parameter can be set either to ENABLE or DISABLE */
68  } NVIC_InitTypeDef;
```

- Complete the interrupt service program in “stm32f10x_it.c”.

Attention:the name of the interrupt service program ！ ！

```
__Vectors       DCD     __initial_sp               ; Top of Stack
                DCD     Reset_Handler              ; Reset Handler
                DCD     NMI_Handler                ; NMI Handler
                DCD     HardFault_Handler          ; Hard Fault Handler
                DCD     MemManage_Handler          ; MPU Fault Handler
                DCD     BusFault_Handler           ; Bus Fault Handler
                DCD     UsageFault_Handler         ; Usage Fault Handler
                DCD     0                          ; Reserved
                DCD     0                          ; Reserved
                DCD     0                          ; Reserved
                DCD     0                          ; Reserved
                DCD     SVC_Handler                ; SVCall Handler
                DCD     DebugMon_Handler           ; Debug Monitor Handler
                DCD     0                          ; Reserved
                DCD     PendSV_Handler             ; PendSV Handler
                DCD     SysTick_Handler            ; SysTick Handler

                ; External Interrupts
                DCD     WWDG_IRQHandler            ; Window Watchdog
                DCD     PVD_IRQHandler             ; PVD through EXTI Line detect
                DCD     TAMPER_IRQHandler          ; Tamper
                DCD     RTC_IRQHandler             ; RTC
                DCD     FLASH_IRQHandler           ; Flash
                DCD     RCC_IRQHandler             ; RCC
                DCD     EXTI0_IRQHandler           ; EXTI Line 0
                DCD     EXTI1_IRQHandler           ; EXTI Line 1
                DCD     EXTI2_IRQHandler           ; EXTI Line 2
                DCD     EXTI3_IRQHandler           ; EXTI Line 3
                DCD     EXTI4_IRQHandler           ; EXTI Line 4
                DCD     DMA1_Channel1_IRQHandler   ; DMA1 Channel 1
                DCD     DMA1_Channel2_IRQHandler   ; DMA1 Channel 2
                DCD     DMA1_Channel3_IRQHandler   ; DMA1 Channel 3
                DCD     DMA1_Channel4_IRQHandler   ; DMA1 Channel 4
                DCD     DMA1_Channel5_IRQHandler   ; DMA1 Channel 5
                DCD     DMA1_Channel6_IRQHandler   ; DMA1 Channel 6
                DCD     DMA1_Channel7_IRQHandler   ; DMA1 Channel 7
                DCD     ADC1_2_IRQHandler          ; ADC1 & ADC2
                DCD     USB_HP_CAN1_TX_IRQHandler  ; USB High Priority or CAN1 TX
                DCD     USB_LP_CAN1_RX0_IRQHandler ; USB Low  Priority or CAN1 RX0
                DCD     CAN1_RX1_IRQHandler        ; CAN1 RX1
                DCD     CAN1_SCE_IRQHandler        ; CAN1 SCE
                DCD     EXTI9_5_IRQHandler         ; EXTI Line 9..5
                DCD     TIM1_BRK_IRQHandler        ; TIM1 Break
                DCD     TIM1_UP_IRQHandler         ; TIM1 Update
                DCD     TIM1_TRG_COM_IRQHandler    ; TIM1 Trigger and Commutation
                DCD     TIM1_CC_IRQHandler         ; TIM1 Capture Compare
                DCD     TIM2_IRQHandler            ; TIM2
                DCD     TIM3_IRQHandler            ; TIM3
                DCD     TIM4_IRQHandler            ; TIM4
```

Example--LED_BasicTImer

```
// 设置中断组为0
NVIC_PriorityGroupConfig(NVIC_PriorityGroup_0);
```

```
// 中断优先级配置
static void BASIC_TIM_NVIC_Config(void)
{
    NVIC_InitTypeDef NVIC_InitStructure;

        // 设置中断来源
    NVIC_InitStructure.NVIC_IRQChannel = TIM6_IRQn ;
        // 设置主优先级为 0
    NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0;
      // 设置抢占优先级为3
    NVIC_InitStructure.NVIC_IRQChannelSubPriority = 3;
    NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
    NVIC_Init(&NVIC_InitStructure);
}
```

```
void TIM6_IRQHandler(void)
{
    if ( TIM_GetITStatus( TIM6, TIM_IT_Update) != RESET )
    {
        time++;
        TIM_ClearITPendingBit(TIM6 , TIM_FLAG_Update);
    }
}
```

main.c

```
/*********************************
芯片：STM32F103ZET6
实现功能：LED灯闪烁（低电平点亮，高电平熄灭），通过基础定时器TIM6来实现1ms定时
引脚：PB5
*********************************/

#include "stm32f10x.h"
#include "led.h"
#include "timer_time.h"

uint16_t time=0;

int main(void)
{
      static uint8_t i=0;
      LED_GPIO_Config();
      // 设置中断组为0
      NVIC_PriorityGroupConfig(NVIC_PriorityGroup_0);

      BASIC_TIM_Init();

      while(1)
        {
              if( time == 1000 )
              {
                  time = 0;
                  switch(i)
                  {
                    case 0:LED_ON();i++;break;
                    case 1:LED_OFF();i=0;break;

                  }
              }
        }
}
```

Homework of lecture5

- ✓ Sort out the process of NVIC configuration; Try to delete the relevant code in the example and write it yourself in Project "LED_BasicTImer";
- ✓ Modify the project"定时器中断（需修改）"，run it in your board. Record what you have modified.

Adrress

4.1 About the STM32 core peripherals

The address map of the *Private peripheral bus* (PPB) is:

Table 33. STM32 core peripheral register regions

Address	Core peripheral	Description
0xE000E010-0xE000E01F	System timer	*Table 49 on page 154*
0xE000E100-0xE000E4EF	Nested vectored interrupt controller	*Table 44 on page 128*
0xE000ED00-0xE000ED3F	System control block	*Table 48 on page 149*
0xE000ED90-0xE000ED93	Memory protection unit	*Table 40 on page 117*(1)
0xE000EF00-0xE000EF03	Nested vectored interrupt controller	*Table 44 on page 128*

https://www.bilibili.com/read/cv12681519

```
/** @addtogroup CMSIS_CM3_NVIC CMSIS CM3 NVIC
  memory mapped structure for Nested Vectored Interrupt Controll
  @{
 */
typedef struct
{
  __IO uint32_t ISER[8];                      /*!< Offset: 0x000
       uint32_t RESERVED0[24];
  __IO uint32_t ICER[8];                      /*!< Offset: 0x080
       uint32_t RSERVED1[24];
  __IO uint32_t ISPR[8];                      /*!< Offset: 0x100
       uint32_t RESERVED2[24];
  __IO uint32_t ICPR[8];                      /*!< Offset: 0x180
       uint32_t RESERVED3[24];
  __IO uint32_t IABR[8];                      /*!< Offset: 0x200
       uint32_t RESERVED4[56];
  __IO uint8_t  IP[240];                      /*!< Offset: 0x300
       uint32_t RESERVED5[644];
  __O  uint32_t STIR;                         /*!< Offset: 0xE00
} NVIC_Type;
/*@}*/ /* end of group CMSIS_CM3_NVIC */
```

4.3.11 NVIC register map

The table provides shows the NVIC register map and reset values. The base address of the main NVIC register block is 0xE000E100. The NVIC_STIR register is located in a separate block at 0xE000EF00.

Table 44. NVIC register map and reset values

Offset	Register	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x000	NVIC_ISER0	SETENA[31:0]																															
	Reset Value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x004	NVIC_ISER1	SETENA[63:32]																															
	Reset Value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x008	NVIC_ISER2	Reserved															SETENA [80:64]															
	Reset Value																0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x080	NVIC_ICER0	CLRENA[31:0]																															
	Reset Value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x084	NVIC_ICER1	CLRENA[63:32]																															
	Reset Value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x088	NVIC_ICER2	Reserved															CLRENA [80:64]															
	Reset Value																0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x100	NVIC_ISPR0	SETPEND[31:0]																															
	Reset Value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x104	NVIC_ISPR1	SETPEND[63:32]																															
	Reset Value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x108	NVIC_ISPR2	Reserved															SETPEND [80:64]															
	Reset Value																0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x180	NVIC_ICPR0	CLRPEND[31:0]																															
	Reset Value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x184	NVIC_ICPR1	CLRPEND[63:32]																															
	Reset Value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x188	NVIC_ICPR2	Reserved															CLRPEND [80:64]															
	Reset Value																0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x200	NVIC_IABR0	ACTIVE[31:0]																															
	Reset Value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0